

Unit 4: Hash Tables

4.1 What is a Hash Table?

A **hash table** (also known as a hash map, dictionary, or associative array) is a data structure that implements an associative array abstract data type, a structure that can map keys to values. A hash table uses a **hash function** to compute an index into an array of buckets or slots, from which the desired value can be found.

Think of it like a super-fast lookup table. Instead of searching through a list ($O(n)$) or even a sorted list ($O(\log n)$), a hash table can often find what you're looking for in $O(1)$ time on average.

Key Components:

- **Keys:** The unique identifiers you use to store and retrieve data (e.g., a person's name, a product ID).
- **Values:** The data associated with the keys (e.g., a person's phone number, a product description).
- **Hash Function:** A function that takes a key and converts it into an index (an integer) in the underlying array.
- **Array (Buckets/Slots):** The actual storage where the values are kept.

4.2 Hash Functions and Collisions

Hash Function

A good hash function should:

- **Be consistent:** Given the same input key, it should always produce the same output index.
- **Map keys to array indices:** The output should be a valid index within the hash table's array.
- **Distribute keys evenly:** It should spread the keys out across the array to minimize collisions.
- **Be fast to compute:** The process of hashing a key should be quick.

Collisions

A **collision** occurs when two different keys hash to the same index in the array. Since a hash table has a finite number of slots, and there are potentially infinite keys, collisions are inevitable.

Collision Resolution Strategies:

1. **Chaining:** Each slot in the array points to a linked list (or another data structure) of key-value pairs that hash to that slot. When a collision occurs, the new key-value pair is simply added to the linked list at that index.
2. **Open Addressing:** When a collision occurs, the algorithm probes for another empty slot in the array. Common probing methods include:
 - **Linear Probing:** Check the next slot, then the next, and so on.
 - **Quadratic Probing:** Check slots at increasing quadratic offsets.
 - **Double Hashing:** Use a second hash function to determine the step size for probing.

Python's dictionaries typically use a sophisticated combination of open addressing and other techniques to handle collisions efficiently.

4.3 How Hash Tables Work (Insertion, Retrieval, Deletion)

Let's consider a simple hash table using chaining.

Insertion (Adding a Key-Value Pair)

1. Take the `key`.
2. Pass the `key` to the `hash function` to get an `index`.
3. Go to that `index` in the array.
4. Add the `(key, value)` pair to the linked list at that `index`.

Retrieval (Getting a Value by Key)

1. Take the `key`.
2. Pass the `key` to the `hash function` to get an `index`.
3. Go to that `index` in the array.
4. Traverse the linked list at that `index` to find the `key`. If found, return its associated `value`.
5. If the `key` is not found in the linked list, it means the key is not in the hash table.

Deletion (Removing a Key-Value Pair)

1. Take the `key`.
2. Pass the `key` to the `hash function` to get an `index`.
3. Go to that `index` in the array.

4. Traverse the linked list at that `index` to find the `key`. If found, remove the `(key, value)` pair from the linked list.

4.4 Hash Tables in Python (Dictionaries)

In Python, hash tables are implemented using the built-in `dict` type. Python dictionaries are incredibly powerful and optimized.

```
# Creating a dictionary
phone_book = {}

# Insertion
phone_book["Alice"] = "555-1234"
phone_book["Bob"] = "555-5678"
phone_book["Charlie"] = "555-9012"
phone_book["Alice"] = "555-1111"

print("Phone book:", phone_book)

# Retrieval
alice_number = phone_book["Alice"]
print(f"Alice's number: {alice_number}")

# Checking if a key exists
if "Bob" in phone_book:
    print("Bob is in the phone book.")

# Deletion
del phone_book["Charlie"]
print("Phone book after deleting Charlie:", phone_book)

# Iterating through a dictionary
print("\nAll entries:")
for name, number in phone_book.items():
    print(f"{name}: {number}")
```

4.5 Big O Notation of Hash Tables

Average Case: $O(1)$ - Constant Time

On average, hash table operations (insertion, retrieval, deletion) take constant time. This is because a good hash function distributes keys evenly, so each bucket contains only a few items. Finding an item then involves:

1. Calculating the hash ($O(1)$).
2. Accessing the array at the calculated index ($O(1)$).
3. Traversing a very short linked list (effectively $O(1)$).

Worst Case: $O(n)$ - Linear Time

The worst-case scenario occurs when all keys hash to the same index (a very bad hash function or a malicious attack). In this case, the hash table degenerates into a single linked list, and operations become $O(n)$ because you have to traverse the entire list.

Factors affecting performance:

- **Load Factor:** The ratio of the number of items in the hash table to the number of available slots. A high load factor increases the probability of collisions.
- **Quality of Hash Function:** A poor hash function leads to more collisions and worse performance.

Key Takeaways

- Hash tables provide incredibly fast average-case lookups ($O(1)$).
- They use a **hash function** to map keys to array indices.
- **Collisions** are handled using strategies like chaining or open addressing.
- Python's `dict` type is an implementation of a hash table.
- Good hash functions and managing the **load factor** are crucial for optimal performance.

end of unit